

Sunbelt PM System V1

Technical Architecture & Integration Guide

Prepared by	Matthew McDaniel
Email	matthew.mcdaniel@sunbeltmodular.com
Date	January 17, 2026
Audience	Joy Thomas & IT Team
Purpose	Technical evaluation, security review, and integration planning

Executive Technical Summary

Metric	Value
Frontend	React 18 + Vite (SPA)
Backend	Supabase (PostgreSQL, Auth, Storage, Edge Functions)
Codebase	~15,000+ LOC across 100+ components
Database	30+ tables with Row Level Security
Hosting Cost	~\$50/month (Supabase Pro)
Security	RLS, JWT auth, input sanitization
npm audit	Clean (no known vulnerabilities)

Technology Stack

Frontend

Technology	Purpose
React 18	UI framework with hooks
Vite	Build tool and dev server
CSS Variables	Theming (dark mode, Sunbelt Orange #FF6B35)
Lucide React	Icon library
React Flow	Workflow visualization
Framer Motion	Animations
PixiJS v8	Factory map (WebGL)
SheetJS	Excel export
PapaParse	CSV parsing
date-fns	Date utilities

Backend (Supabase)

Service	Purpose
PostgreSQL	Primary database
Auth	Email/password + JWT
Storage	File uploads (floor plans, documents)
Edge Functions	Server-side logic (Deno)
Realtime	Live subscriptions (planned)

PWA Technologies

Technology	Purpose
Service Worker	Offline support
Workbox	Caching strategies
Web App Manifest	Install capability

System Architecture



Database Schema Overview

Core Tables

Table	Purpose	Key Fields
users	User accounts	id, email, role, factory_id
factories	Factory locations	id, name, code, timezone
projects	Project records	id, name, status, factory_id, owner_id, praxis_quote_number
tasks	Task tracking	id, project_id, status, priority, assigned_by, due_date
rfis	RFI management	id, project_id, subject, status, question
submittals	Submittal tracking	id, project_id, status, spec_section

Production Tables

Table	Purpose	Key Fields
modules	Production modules	id, serial_number, project_id, current_station_id, status
station_templates	Station definitions	id, factory_id, name, sequence
workers	Factory workers	id, employee_id, factory_id, is_lead, pin_hash
worker_shifts	Time tracking	id, worker_id, clock_in, clock_out, break_minutes

Table	Purpose	Key Fields
qc_records	QC inspections	id, module_id, station_id, passed, defect_codes

Sales Tables

Table	Purpose	Key Fields
sales_quotes	Quote lifecycle	id, quote_number, status, outlook_percentage
dealers	Dealer database	id, name, contact_info
praxis_import_log	Import tracking	id, filename, row_count, status

Support Tables

Table	Purpose
floor_plans	Floor plan files and markers
floor_plan_markers	Coordinate markers on plans
project_documents_checklist	Required document tracking
long_lead_items	Long lead item tracking
workflow_stations	Project workflow state
system_errors	Error logging
announcements	System announcements
feature_flags	Feature toggles

Security Implementation

Authentication

Method	Implementation
Desktop App	Email/password via Supabase Auth
Floor PWA	PIN-based via Edge Function (bcrypt)
Manager PWA	Email/password via Supabase Auth
Session	JWT tokens (15-min expiry, auto-refresh)

Authorization (Row Level Security)

All tables have RLS policies enforcing:

```
-- Example: Users see only their factory's data
CREATE POLICY "factory_isolation" ON projects
FOR ALL USING (
  factory_id IN (
    SELECT factory_id FROM users WHERE id = auth.uid()
  )
);

-- Example: Role-based access
CREATE POLICY "pm_update_own_projects" ON projects
FOR UPDATE USING (
  owner_id = auth.uid() OR
  primary_pm_id = auth.uid() OR
  backup_pm_id = auth.uid()
);
```

Role Hierarchy

Role	Access Level
VP	All factories, all data (read), pipeline visibility
Director	Assigned factories, team projects
PM	Own projects, factory data
PC	Factory projects, deadline focus
Plant Manager	Factory production, crew, modules
Sales Manager	Team quotes, pipeline
Sales Rep	Own quotes only
IT Manager	Admin tools, error logs, feature flags

Data Protection

Layer	Implementation
Input Sanitization	DOMPurify for XSS prevention
SQL Injection	Parameterized queries (Supabase SDK)
HTTPS	All traffic encrypted
Storage	Private buckets, signed URLs
Secrets	Environment variables only

Application Components Inventory

Desktop Dashboards (5)

Dashboard	File	LOC	Features
VPDashboard	src/components/dashboards/VPDashboard.jsx	~400	Portfolio, pipeline, factory metrics
DirectorDashboard	src/components/dashboards/DirectorDashboard.jsx	~500	Timeline, PM workload, health tracking
PMDashboard	src/components/dashboards/PMDashboard.jsx	~350	My tasks, calendar, deliveries
PlantManagerDashboard	src/components/dashboards/PlantManagerDashboard.jsx	~800	9 tabs, production, crew, QC
PCDashboard	src/components/dashboards/PCDashboard.jsx	~300	Deadlines, overdue, factory projects

Production Components (15+)

Component	Purpose
TaktTimeTracker	Cycle time by station
QueueTimeMonitor	Queue wait analysis
OEETCalculator	Overall Equipment Effectiveness
VisualLoadBoard	Module distribution
KaizenBoard	Improvement tracking

Component	Purpose
DefectCycleTimer	Defect aging
AttendanceDashboard	Clock in/out, breaks
CrewScheduleView	Weekly assignments
CrewUtilizationHeatmap	Efficiency visualization
CrossTrainingMatrix	Skills tracking
QCInspectionModal	Pass/fail form
StationDetailModal	Station info
ModuleDetailModal	Module info
ProductionCalendar	Scheduling
SimModeToolbar	Simulation controls

Project Components (20+)

Component	Purpose
ProjectDetails	Full project view (6+ tabs)
WorkflowCanvas	Interactive workflow
TasksView	Task management
FloorPlansTab	Floor plan viewer
AddTaskModal / EditTaskModal	Task CRUD
AddRFIModal / EditRFIModal	RFI CRUD
AddSubmittalModal / EditSubmittalModal	Submittal CRUD
PraxisImportModal	CSV/Excel import

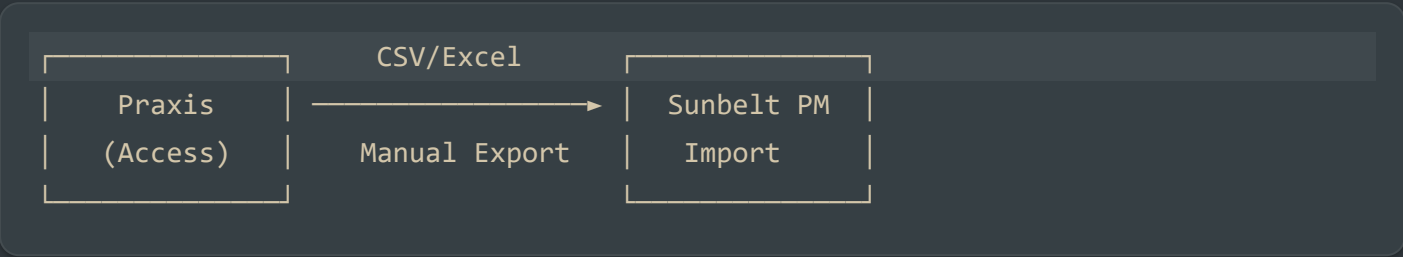
Component	Purpose
RFILogExport	Excel export

PWA Components

App	Components	Purpose
Floor PWA	PWAHome, ModuleLookup, StationMove, QCInspection	Worker interface
Manager PWA	ManagerDashboard, ProjectsList, TasksView, RFIsView	Supervisor interface

Praxis Integration

Current State (Level 1 - File-Based)



Implementation:

- Export from Praxis as CSV/Excel
- Upload via PraxisImportModal
- PapaParse/SheetJS parsing
- ~40 field mappings normalized
- Insert to `sales_quotes` or `projects`

Field Mappings (Sample):

Praxis Field	Database Column
Quote Number	praxis_quote_number
Building Length	building_length
Building Width	building_width
Material Cost	material_cost
Customer Name	client_name
Dealer	dealer_id (FK)
Factory	factory_id (FK)

Future Options

Level 2: API Wrapper

- Local Node.js server with ODBC connection
- Scheduled sync (every 15 min)
- Supabase Edge Function for processing

Level 3: Full Integration

- Real-time sync via triggers
- Bi-directional updates
- Conflict resolution logic

Hosting & Infrastructure

Current Setup

Component	Service	Cost
Frontend	Vercel/Netlify (static)	Free tier

Component	Service	Cost
Backend	Supabase Pro	\$25/month
Storage	Supabase Storage	Included
Edge Functions	Supabase Edge	Included
Total		~\$25-50/month

Scaling Path

Users	Recommendation	Est. Cost
1-50	Supabase Pro	\$25/month
50-100	Supabase Pro	\$50/month
100-200	Supabase Pro + compute add-on	\$75-100/month
200+	Supabase Enterprise or self-host	Custom

Network Requirements

Requirement	Details
Protocol	HTTPS (443)
Bandwidth	Minimal (~50KB per page load)
Latency	<200ms recommended
Firewall	Allow *.supabase.co

Performance Optimizations

Database

Optimization	Implementation
Indexes	Composite indexes on frequently queried columns
Full-text Search	GIN indexes for search (projects, tasks, RFIs)
Query Optimization	EXPLAIN ANALYZE on slow queries
Connection Pooling	Supabase built-in (PgBouncer)

Frontend

Optimization	Implementation
Code Splitting	React.lazy() for route-based splitting
Memoization	useMemo, useCallback for expensive operations
Skeleton Loading	Loading states for better UX
Debouncing	Search/filter inputs

SQL Scripts Available

Script	Purpose
<code>PERFORMANCE_INDEXES.sql</code>	Create performance indexes
<code>AUDIT_STATUS_VALUES.sql</code>	Audit status consistency
<code>AUDIT_RLS_POLICIES.sql</code>	Verify RLS coverage

Error Handling & Monitoring

Current Implementation

Layer	Approach
React	ErrorBoundary component with fallback UI
API Calls	Try/catch with user-friendly messages
Database	Error logging to <code>system_errors</code> table
Validation	<code>src/utils/errorHandling.js</code> utilities

Error Utilities

```
// src/utils/errorHandling.js
handleSupabaseError(error)    // Maps technical errors to user messages
withErrorHandling(asyncFn)    // Wrapper for async operations
validateRequired(data, fields) // Pre-validation
createResult(success, data, error) // Standardized response
```

Planned Enhancements

Enhancement	Purpose
Sentry	Error tracking and replay
Uptime Monitoring	Service availability
Performance Metrics	Core Web Vitals

Deployment Process

Current Workflow

1. Local Development (Vite dev server)
 2. Git commit/push to main
 3. Vercel/Netlify auto-deploy
 4. Supabase migrations via CLI

Recommended CI/CD

```
# .github/workflows/deploy.yml
- Run npm audit
- Run ESLint
- Run tests (when available)
- Build production bundle
- Deploy to staging
- Manual approval → Production
```

Mobile PWA Technical Details

Floor App (Worker PWA)

Feature	Technical Implementation
PIN Auth	bcrypt via Edge Function
Module Lookup	Supabase query + camera scan (future)
Station Move	Update module.current_station_id
QC Inspection	Insert to qc_records
Offline	Service worker + IndexedDB cache

Manager PWA

Feature	Technical Implementation
Auth	Supabase Auth (shared with desktop)
Projects	Paginated queries with filters
Create Task/RFI	Bottom sheet modals → Supabase insert
Sync Indicator	Online/offline detection

IT Collaboration Opportunities

Immediate (Post-Demo)

Item	Description
Code Review	Access to GitHub repo for security audit
Pilot Setup	Configure for 1 factory
User Provisioning	Create initial user accounts
Network Access	Whitelist Supabase endpoints

Short-Term

Item	Description
SSO Integration	SAML/OAuth with corporate identity
Monitoring	Add to existing monitoring stack
Backup Strategy	Configure database backup retention

Long-Term

Item	Description
Praxis API	Build ODBC wrapper for automated sync
On-Prem Option	Evaluate self-hosted Supabase
Mobile MDM	PWA deployment via MDM

Anticipated Technical Questions

"What are the infrastructure requirements?"

Browser-based SPA with cloud backend. No servers to maintain. Requires HTTPS access to *.supabase.co domains.

"How is data isolated between factories?"

Row Level Security (RLS) on every table. Users can only query data for their assigned factory. Enforced at database level.

"What happens if Supabase goes down?"

Supabase has 99.9% uptime SLA. For critical uptime, we can self-host PostgreSQL with their open-source stack.

"How do we handle data conflicts?"

Timestamps track last-modified. Import logs track all Praxis syncs. Conflict resolution uses latest-wins or manual review.

"Can we audit user actions?"

Yes. `system_errors` table logs errors. Activity logging can be added via database triggers or Edge Functions.

"What's the backup strategy?"

Supabase provides daily backups (7-day retention on Pro). Point-in-time recovery available. Additional backups can be configured.

"Is the code maintainable?"

Yes. Clean folder structure, consistent patterns, comprehensive documentation. AI-assisted development with code review.

Repository Structure

```
sunbelt-pm-system-v1/
├── src/
│   ├── components/
│   │   ├── auth/           # Login, signup
│   │   ├── calendar/      # Calendar views
│   │   ├── common/        # Shared components
│   │   ├── dashboards/    # Role dashboards
│   │   ├── factoryMap/    # PixiJS map
│   │   ├── floorplans/    # Floor plan viewer
│   │   ├── it/            # Admin tools
│   │   ├── production/    # Production components
│   │   ├── projects/      # Project management
│   │   ├── sales/         # Sales/quotes
│   │   └── workflow/      # Workflow visualization
│   ├── context/           # React contexts
│   ├── hooks/             # Custom hooks
│   ├── pwa/               # PWA applications
│   │   ├── floor/         # Worker app
│   │   └── manager/       # Manager app
│   ├── services/          # API service functions
│   ├── styles/            # CSS variables
│   └── utils/              # Utility functions
├── supabase/
│   ├── migrations/        # Database migrations
│   ├── functions/         # Edge Functions
│   └── demo/              # Demo data scripts
```

└ docs/	# Documentation
└ public/	# Static assets

Next Steps for IT

1. **Review** - Access GitHub repo for code review
2. **Evaluate** - Test demo environment
3. **Plan** - Identify pilot factory and users
4. **Configure** - Set up production Supabase project
5. **Deploy** - Deploy frontend to Vercel/Netlify
6. **Monitor** - Add to monitoring stack

Contact & Resources

Resource	Location
GitHub Repo	[To be shared with IT]
Demo Environment	[Deployed URL]
Documentation	/docs folder in repo
Technical Contact	Matthew McDaniel (matthew.mcdaniel@sunbeltmodular.com)

Sunbelt PM System V1 - Technical Documentation
Built with React + Supabase
Sunbelt Orange: #FF6B35